

Documento di Design – Gestione Biblioteche Romane

1. Introduzione

2. Obiettivi del Documento

- Fornire una panoramica dell'architettura del sistema.
- Descrivere in dettaglio i moduli e le componenti del sistema.
- Definire le interfacce e le modalità di comunicazione tra le componenti.
- Esplicitare le scelte progettuali e i pattern di design adottati.
- Costituire un riferimento per le fasi di implementazione e validazione.

3. Architettura del Sistema

Il sistema è strutturato secondo un'architettura a strati, che separa le responsabilità in tre livelli principali:

- **Presentazione (Front-End):** Interfaccia utente web responsive per la gestione delle operazioni (ricerca libri, autenticazione, prenotazioni, ecc.).
- **Business Logic (Back-End):** Moduli responsabili della gestione delle operazioni e delle regole di business, organizzati per ambiti funzionali (autenticazione, prestito, gamification, ecc.).
- **Persistenza:** Gestione e conservazione dei dati (utenti, libri, prestiti, tessere, prenotazioni, ecc.) tramite un database relazionale.

La comunicazione tra i livelli avviene tramite API RESTful, mentre l'integrazione con servizi esterni (es. SPID, CIE) è gestita tramite interfacce specifiche e webhook sicuri.

4. Design delle Componenti

Il sistema è suddiviso in moduli che rispecchiano i casi d'uso e le funzionalità identificate nell'analisi:

4.1 Modulo Autenticazione

- **Funzioni:**
 - Gestione del login tramite sistema interno ed esterno (AU_1, AU_2).
 - Registrazione e modifica della password (AU_3, AU_4).
 - Logout (AU_5).
- **Design:**
 - Utilizzo di form sicuri e validazione lato client e server.
 - Integrazione con sistemi di autenticazione esterni tramite protocolli standard.

4.2 Modulo Tesseramento

- **Funzioni:**
 - Nuovo tesseramento, rinnovo, cancellazione, denuncia di smarrimento e furto (T_1, T_2, T_3, T_4, T_5).
- **Design:**
 - Gestione tramite form di richiesta con validazione dei dati.

- Interazione con il modulo amministrativo per approvazione e aggiornamento dello stato della tessera.

4.3 Modulo Gamification

- **Funzioni:**
 - Partecipazione, gestione profilo, inserimento e segnalazione di commenti/recensioni, riscossione dei punti (G_1, G_2, G_3, G_4, G_5).
- **Design:**
 - Interfaccia dedicata per la visualizzazione dei punteggi e dei premi.
 - Meccanismi di moderazione e segnalazione per garantire la qualità dei contenuti.

4.4 Modulo Prestito Libri

- **Funzioni:**
 - Ricerca e richiesta di prestito di libri fisici ed e-book (PL_1, PL_4).
 - Restituzione e richiesta di allungamento del prestito (PL_2, PL_3).
- **Design:**
 - Catalogo dinamico con paginazione per gestire grandi volumi di dati.
 - Notifiche via email o sistema di alert per la restituzione o l'allungamento.

4.5 Modulo Prenotazione Spazi

- **Funzioni:**
 - Prenotazione e cancellazione di spazi comuni (PS_1, PS_2).
 - Segnalazione di comportamenti errati (PS_3).
- **Design:**
 - Visualizzazione in tempo reale della disponibilità degli spazi.
 - Sistema di conferma e notifica delle prenotazioni tramite email.

4.6 Modulo Sezione Amministrativa

- **Funzioni:**
 - Caricamento nuovi libri, segnalazione prestiti e pubblicazione di notizie (SA_1, SA_2, SA_3, SA_4).
- **Design:**
 - Dashboard amministrativa con accesso riservato.
 - Funzionalità di editing e aggiornamento centralizzato dei contenuti.

5. Interfacce e Comunicazione

- **API RESTful:**

Le componenti del sistema comunicano tramite API RESTful. Ogni modulo espone endpoint per le operazioni CRUD e per la gestione dei flussi di lavoro.
- **Interfacce Utente:**

Il front-end fornisce un'interfaccia intuitiva e responsive, sviluppata utilizzando framework moderni, e interagisce con il back-end tramite chiamate API.

- **Integrazione Esterna:**

Le comunicazioni con servizi esterni (es. SPID, CIE) sono gestite tramite appositi client e middleware, garantendo la sicurezza e l'affidabilità dei webhook.

6. Design delle Classi e Pattern

Il design orientato agli oggetti è supportato dalle schede CRC sviluppate precedentemente. Le classi principali e i loro metodi sono definiti seguendo il principio nomi-verbi, per garantire chiarezza e coerenza. Tra i pattern e le scelte progettuali adottate troviamo:

- **MVC (Model-View-Controller):** Per separare la logica di presentazione dalla business logic.
- **Repository Pattern:** Per l'astrazione della persistenza e la gestione delle operazioni CRUD.
- **Dependency Injection:** Per gestire le dipendenze tra le componenti e facilitare il testing.
- **Singleton:** Per gestire risorse condivise, come la connessione al database o i client per i servizi esterni.

Le CRC cards (consultabili nel documento di analisi) definiscono in modo dettagliato le responsabilità di ciascuna classe e i collaboratori con cui interagiscono.

7. Scelte Tecnologiche

- **Front-End:**
 - Linguaggio: JavaScript/TypeScript.
 - Framework: Vue.js.
 - Librerie: Bootstrap per l'interfaccia utente.
- **Back-End:**
 - Linguaggio: Go
 - Framework: -
 - API RESTful per la comunicazione tra front-end e back-end.
- **Persistenza:**
 - Database relazionale (PostgreSQL)
- **Sicurezza:**
 - Autenticazione e autorizzazione basate su token (JWT).
 - Cifratura dei dati sensibili e implementazione di protocolli HTTPS.

8. Considerazioni sul Deployment e Scalabilità

- **Deployment:**
 - Il sistema sarà distribuito su un ambiente cloud per garantire scalabilità e alta disponibilità.
 - Utilizzo di container (Docker) per il packaging e il deployment delle componenti.
- **Scalabilità:**
 - L'architettura a micro-servizi permette di scalare singolarmente i vari componenti in base al carico.
 - Utilizzo di load balancer e sistemi di caching per migliorare le performance.

9. Conclusioni